

Here is your content, professionally formatted for an exam-style presentation while keeping your exact information intact.

Part 1: Databases used in Pass I of Two-Pass Assembler

Introduction

In **Pass 1**, the assembler primarily focuses on defining symbols and literals and determining the storage requirements. It creates and uses the following key data structures:

1. Symbol Table (SYMTAB)

- **Purpose:** Stores symbol names (labels/variables) and their assigned memory addresses.
- **Example:**

Symbol	Address
LOOP	200
A	205
B	210

- **Used to:** Assign addresses and resolve symbols during Pass 2.

2. Literal Table (LITTAB)

- **Purpose:** Stores literals (constants) encountered in the program.
- **Example:**

Literal	Address

=5	300
=10	301

- **Timing:** Addresses are assigned when an LTOrg or END statement is encountered.

3. Pool Table (POOLTAB)

- **Purpose:** Keeps track of literal pools (groups of literals).
- **Example:**

Pool#	Start Index
1	1
2	3

- **Used to:** Group literals and ensure they are assigned addresses correctly.

4. Intermediate Code (IC)

- **Purpose:** The output of Pass 1 and the input for Pass 2.
- **Example:**
 - (AD, START) 100
 - (IS, MOVER) AREG, (S,1)
 - (IS, ADD) BREG, (L,1)
- **Contains:** Operation codes, symbol references, and literal references.

5. Location Counter (LC)

- **Purpose:** A register that tracks the current memory address being processed.
- **Example:** * START 100 $\rightarrow$ LC = 100
 - Next instruction $\rightarrow$ LC = 101

6. Opcode Table (OPTAB / MOT)

- **Purpose:** A permanent database storing mnemonics, instruction types, and lengths.
- **Example:**

- `MOVER $rightrightarrow$` Type: IS, Opcode: 04
- `ADD $rightrightarrow$` Type: IS, Opcode: 01

Part 2: Forward Reference with Example

◆ Definition

A **forward reference** occurs when a symbol is used in an instruction before it has been defined in the program.

◆ Example

Code snippet

```
LOAD 1, X ; Symbol X is used here
...
X DC F'3' ; Symbol X is defined here
```

◆ Explanation

In the first instruction `LOAD 1, X`, the assembler encounters the symbol **X**. However, **X** is defined later in the code using the `DC` statement. Since the definition comes after the usage, it is a forward reference.

◆ Handling the Reference

- **Pass 1:** The assembler does not know the value of **X** yet. it enters **X** into the **SYMTAB** and marks its address once the definition is reached (e.g., $X = 3$).
- **Pass 2:** The assembler retrieves the actual address of **X** from the **SYMTAB** and completes the instruction: `LOAD 1, 3`.

🎯 **Final One-line Answer:** A forward reference is a situation where a symbol is used before its definition, which is resolved during Pass 2 using the Symbol Table.

Part 3: Types of Assembly Language

Statements

1. Imperative Statements (IS)

- **Purpose:** These perform actual operations and are executed by the CPU. They generate machine instructions.
- **Examples:** MOVER AREG, A, ADD BREG, B, SUB CREG, C
- **Explanation:** MOVER moves data, ADD performs addition, and SUB performs subtraction.

2. Declarative Statements (DL)

- **Purpose:** Used to declare variables/constants and allocate memory space.
- **Examples:** * A DS 1 (Define Storage: Reserves 1 word)
 - B DC 5 (Define Constant: Stores the value 5)

3. Assembler Directives (AD)

- **Purpose:** Instructions to the assembler itself; these are **not** executed by the CPU.
- **Examples:** * START 100: Defines starting address.
 - END: Marks the end of the program.
 - ORIGIN 200: Changes the Location Counter.
 - LTORG: Assigns addresses to literals.

◆ Summary Table

Type	Purpose	Example
Imperative (IS)	Perform CPU operations	MOVER, ADD
Declarative (DL)	Define data/memory	DS, DC
Assembler Directive (AD)	Control assembly process	START, END

Here is your content formatted professionally for your document. I have organized it to be clear, readable, and ready to be pasted while keeping your exact descriptions and examples.

Advanced Assembler Directives (Pseudo-Operations)

1. Introduction

- **Assembler Directives (AD)** are instructions given specifically to the assembler to control the assembly process.
- **Advanced directives** provide high-level control over relocation, program linking, symbol definition, and base register management.
- **Crucial Point:** These are **not executed by the CPU**; they are instructions for the compiler/assembler itself.

2. Key Advanced Assembler Directives

A. ORIGIN

- **Purpose:** Manually changes the value of the **Location Counter (LC)**.
- **Example:**
- Code snippet

```
START 100
A DS 1
ORIGIN A + 5
B DS 1
```

-
-
- **Effect:** The LC "jumps" directly to the address `A + 5`.

B. EQU (Equate)

- **Purpose:** Assigns a specific constant value or expression to a symbol.
- **Example:**
- Code snippet

```
X EQU 10
Y EQU X + 5
```

-
-

- **Effect:** `X` is defined as 10, and `Y` becomes 15.

C. LTORG (Literal Origin)

- **Purpose:** Forces the assembler to allocate memory for all literals used so far at that specific point in the program.
- **Example:**
- Code snippet

```
MOVER AREG, =5
ADD BREG, =10
LTORG
```

-
-
- **Effect:** Literals `=5` and `=10` are assigned memory addresses immediately following the `LTORG` statement.

D. START

- **Purpose:** Specifies the starting memory address of the program.
- **Example:** `START 200`
- **Effect:** The Location Counter (LC) is initialized to 200.

E. END

- **Purpose:** Marks the physical end of the program and triggers the assignment of addresses to any remaining literals.

F. USING (Base Register Directive)

- **Purpose:** Informs the assembler which register to use as a **Base Register** for address calculation.
- **Example:** `USING *, 15`
- **Effect:** Tells the assembler to use Register 15 as the base register.

G. DROP

- **Purpose:** Releases a base register previously assigned by a `USING` directive.
- **Example:** `DROP 15`

- **Effect:** Register 15 is no longer used for base-relative addressing.

3. Summary Table

Directive	Purpose
START	Initializes the Location Counter (LC).
END	Marks the end of the program.
ORIGIN	Shifts the LC to a specific address.
EQU	Defines a symbol with a constant value.
LTORG	Assigns addresses to the current literal pool.
USING	Defines a base register for addressing.
DROP	Releases a designated base register.

4. Conclusion

Advanced assembler directives are essential for managing memory allocation, symbol

definitions, and program organization. They provide the flexibility required to write efficient and relocatable assembly code.

🎯 **One-line Exam Answer:** Advanced assembler directives like `ORIGIN`, `EQU`, `LTORG`, `USING`, and `DROP` are pseudo-operations that control memory mapping, symbol value assignment, and register usage during the assembly process.

Here is your content on the **Elements of Assembly Language Programming**, formatted for a professional study guide or document.

Elements of Assembly Language Programming

1. Introduction

- An assembly language program consists of statements that follow a specific, rigid format.
- Each statement is composed of different **elements (fields)** that allow the assembler to translate the human-readable mnemonics into machine code.

2. Format of an Assembly Statement

A typical assembly statement follows this general structure:

`[Label] Opcode Operand(s) ; Comment`

3. Core Elements

A. Label

- **Definition:** A symbolic name used to identify a specific memory location or instruction.
- **Example:** `LOOP: MOVER AREG, A`
- **Purpose:** Primarily used for branching (jumping) and referencing specific lines of code.

B. Opcode (Operation Code)

- **Definition:** Specifies the exact operation the CPU must perform.
- **Examples:** `MOVER`, `ADD`, `SUB`, `MULT`.

- **Purpose:** Represents the fundamental instructions executed by the processor.

C. Operand(s)

- **Definition:** Specifies the data or the memory address on which the operation is performed.
- **Types:**
 - **Register:** AREG, BREG
 - **Memory:** A, B
 - **Constant:** =5
- **Example:** In MOVER AREG, A, both AREG and A are operands.

D. Comment

- **Definition:** Descriptive text used to explain the logic of the code.
- **Example:** ADD AREG, B ; Add value of B to AREG
- **Rule:** Comments are entirely **ignored by the assembler** during translation.

4. Supporting Elements

- **Constants / Literals:** Fixed values used directly in the program (e.g., =5).
- **Symbols:** Names used to represent variables or memory addresses (e.g., A DS 1).
- **Registers:** High-speed CPU storage locations used for arithmetic and data movement (e.g., AREG, BREG).
- **Assembler Directives:** Instructions that provide directions to the assembler rather than the CPU (e.g., START, END).

5. Example Statement Breakdown

Statement: LOOP: ADD AREG, B ; Add B to AREG

Element	Value
Label	LOOP

Opcode	ADD
Operand	AREG, B
Comment	Add B to AREG

6. Conclusion

The structure of an assembly language program is defined by the interaction between labels, opcodes, operands, and directives. Together, these elements ensure the program is both human-readable and accurately translatable for the hardware.

 **One-line Exam Answer:** The primary elements of assembly language programming include the label, opcode, operand, comment, symbols, constants, and assembler directives.
